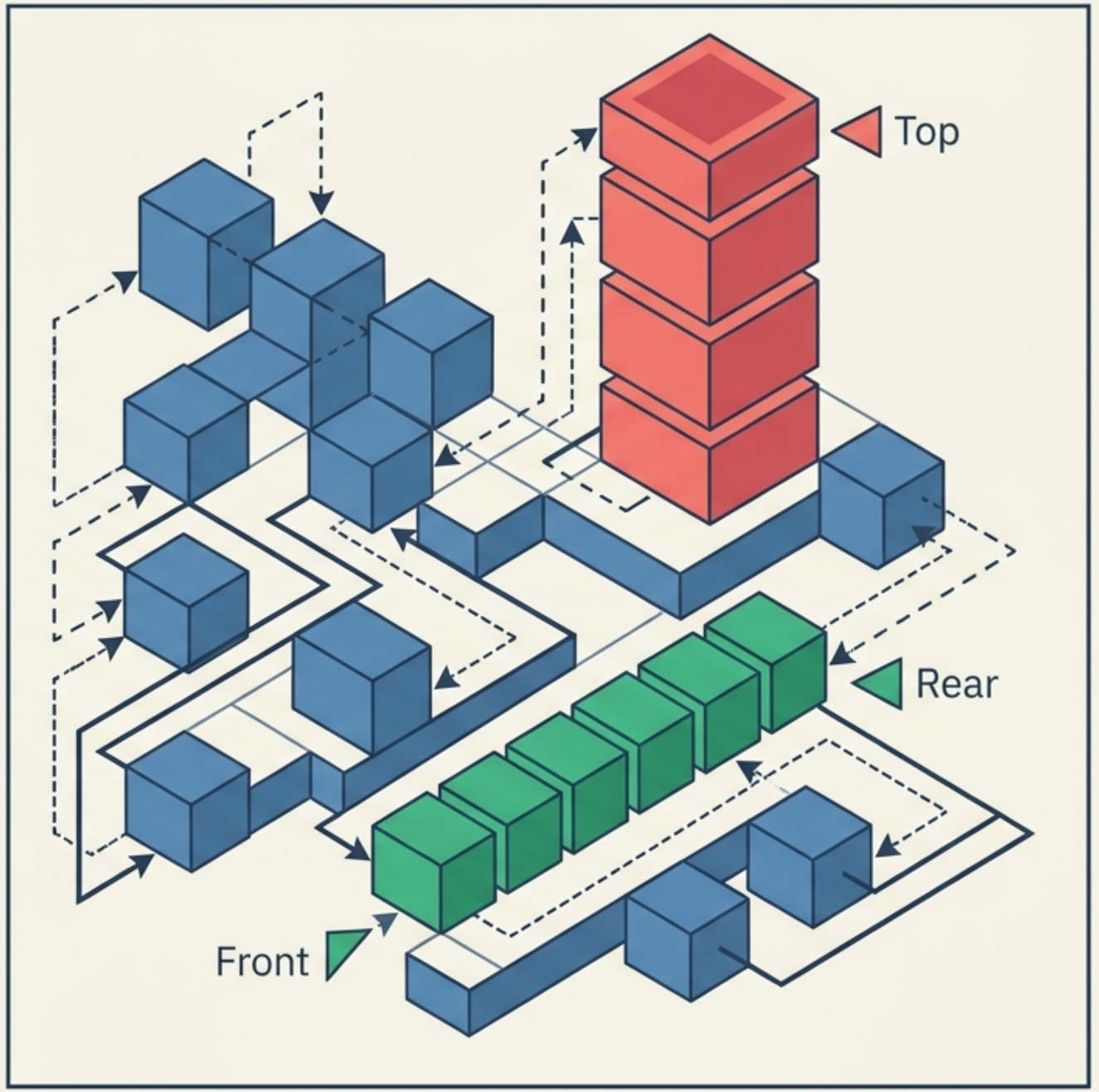
Estructuras de Datos Dinámicas

Dominando la Memoria, las Pilas y las Colas

Arquitectura de software
a nivel de punteros



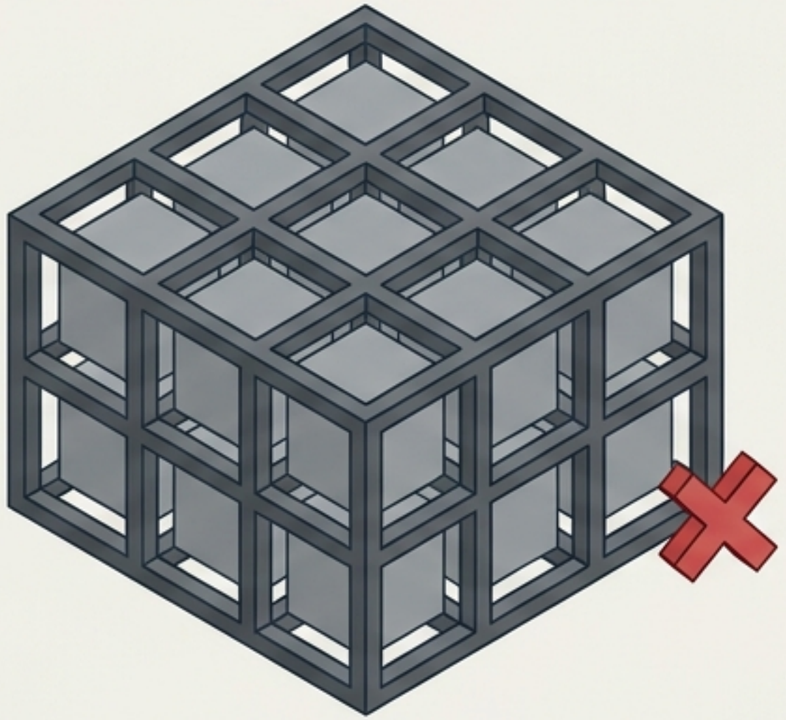
NotebookLM

El Límite de lo Estático

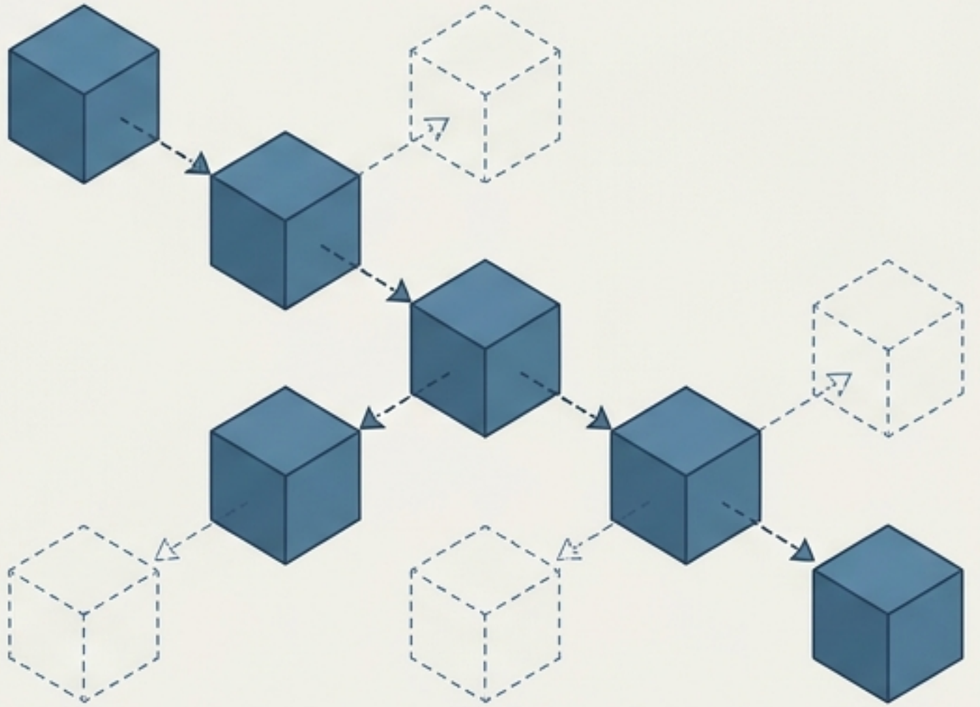
¿Por qué necesitamos estructuras dinámicas?

Los vectores y matrices convencionales tienen un tamaño fijo que no varía. Las estructuras dinámicas pueden crecer o reducirse libremente en tiempo real durante la ejecución del programa.

ESTÁTICO



DINÁMICO

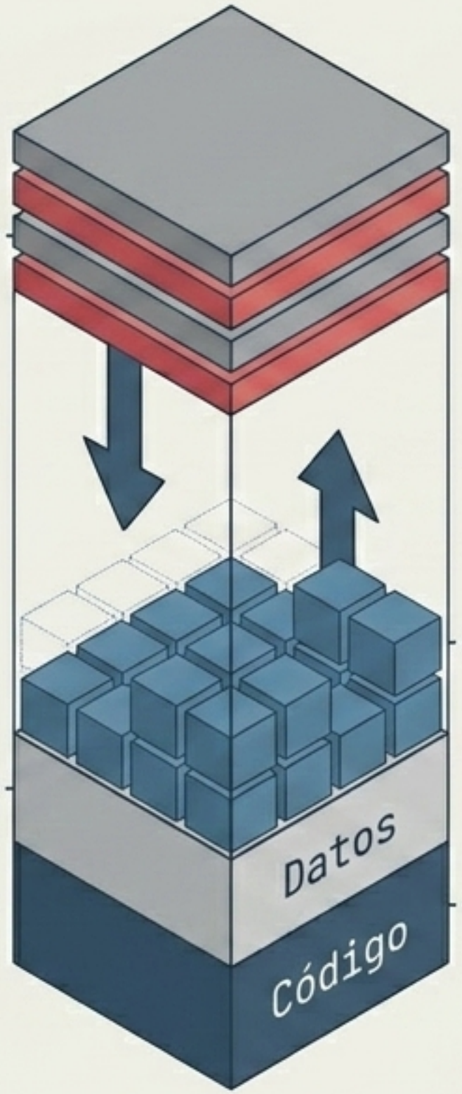


NotebookLM

Anatomía de la Memoria de un Proceso

Stack (Pila)
Apila llamadas a funciones y sus parámetros.

Datos
Variables locales, globales y estáticas.



Heap
Reserva de memoria dinámica. Crece o disminuye bajo demanda.

Código
Contiene el ejecutable del programa.

NotebookLM

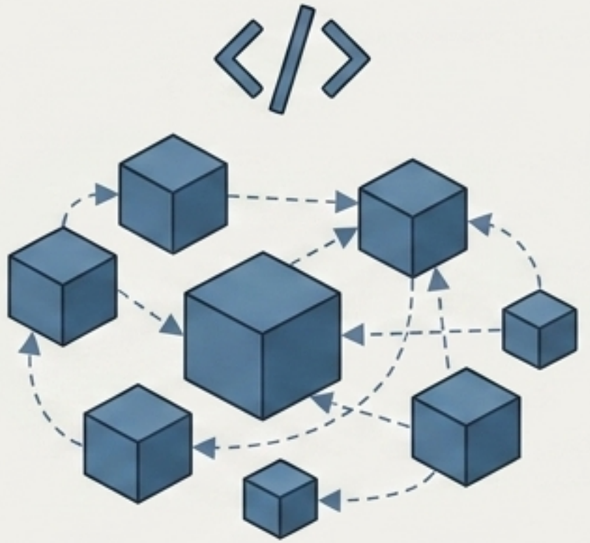
El Duelo: Stack vs. Heap

STACK



- ✓ Gestionado automáticamente por el Sistema Operativo.
- ✓ Orden estricto LIFO.
- ✓ Destrucción automática al retornar la función.

HEAP

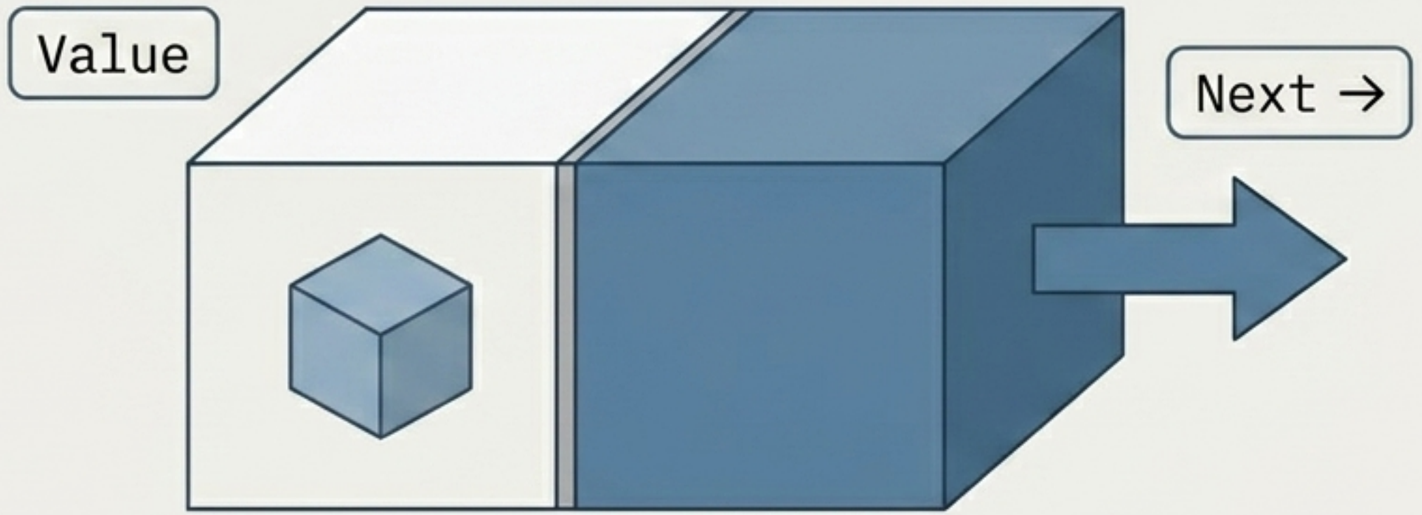


- ✓ Gestionado por el programador (o Garbage Collector).
- ✓ Sin algoritmo predefinido de orden.
- ✓ Hogar de las estructuras dinámicas (Pilas, Colas, Listas).

NotebookLM

El Bloque de Construcción: El Nodo

Las estructuras dinámicas están compuestas por Nodos. Gracias a los genéricos (<T>), un nodo puede almacenar números, texto o cualquier objeto complejo.




```
public class MyNode<T> {
    public T Value { get; }
    public MyNode<T> Next { get; set; }
}
```

NotebookLM

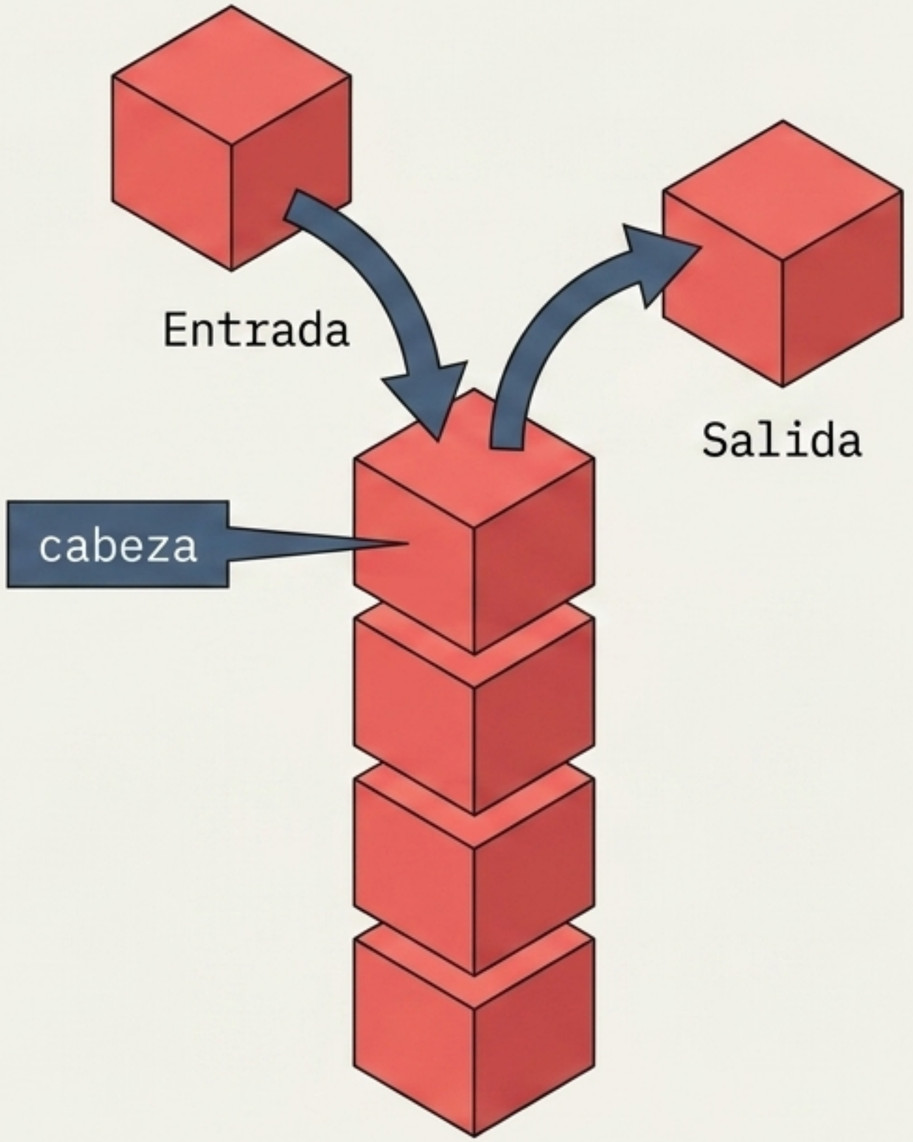
Pilas (Stacks): El Principio LIFO

Orden estricto: Último entrado, primero en salir (Last In, First Out).

Solo se puede acceder, introducir o eliminar elementos por un único lugar denominado cabeza de pila (tope).

CASOS DE USO

- Historial de navegación web
- Función "Deshacer" (Ctrl+Z)



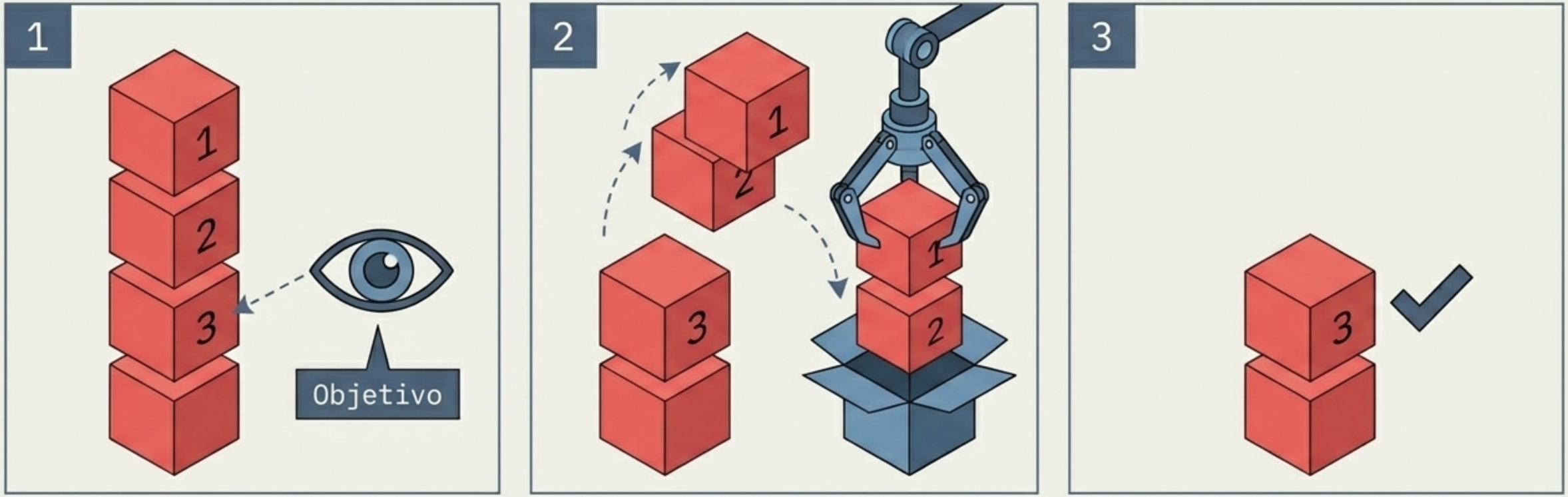
NotebookLM

Advertencia: Lectura Destructiva

A diferencia de un vector, no puedes simplemente “mirar” el medio de una estructura dinámica.

Para leer un elemento, debes removerlo permanentemente usando la función de extracción.

Regla de oro: Siempre verifica `IsEmpty()` antes de extraer para evitar errores de ejecución.

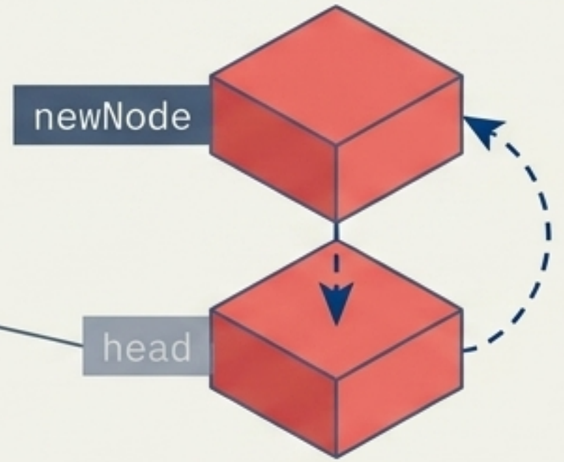


NotebookLM

Operaciones de Pila: Push & Pop

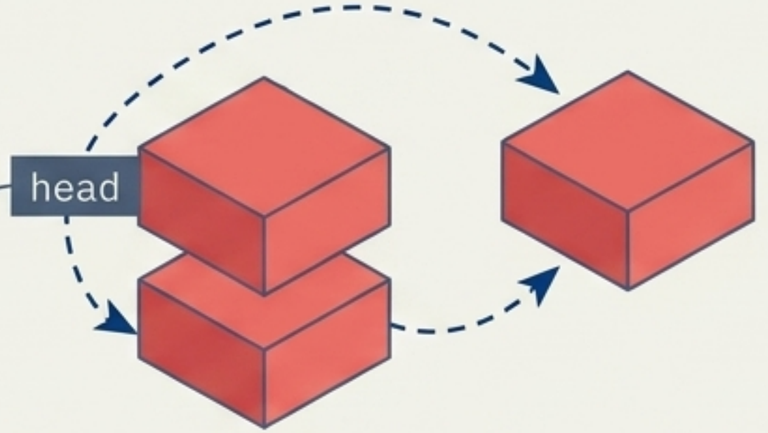
1. PUSH (Insertar)

```
newNode.Next = head;
head = newNode;
```



2. POP (Extraer)

```
T value = head.Value;
head = head.Next;
return value;
```



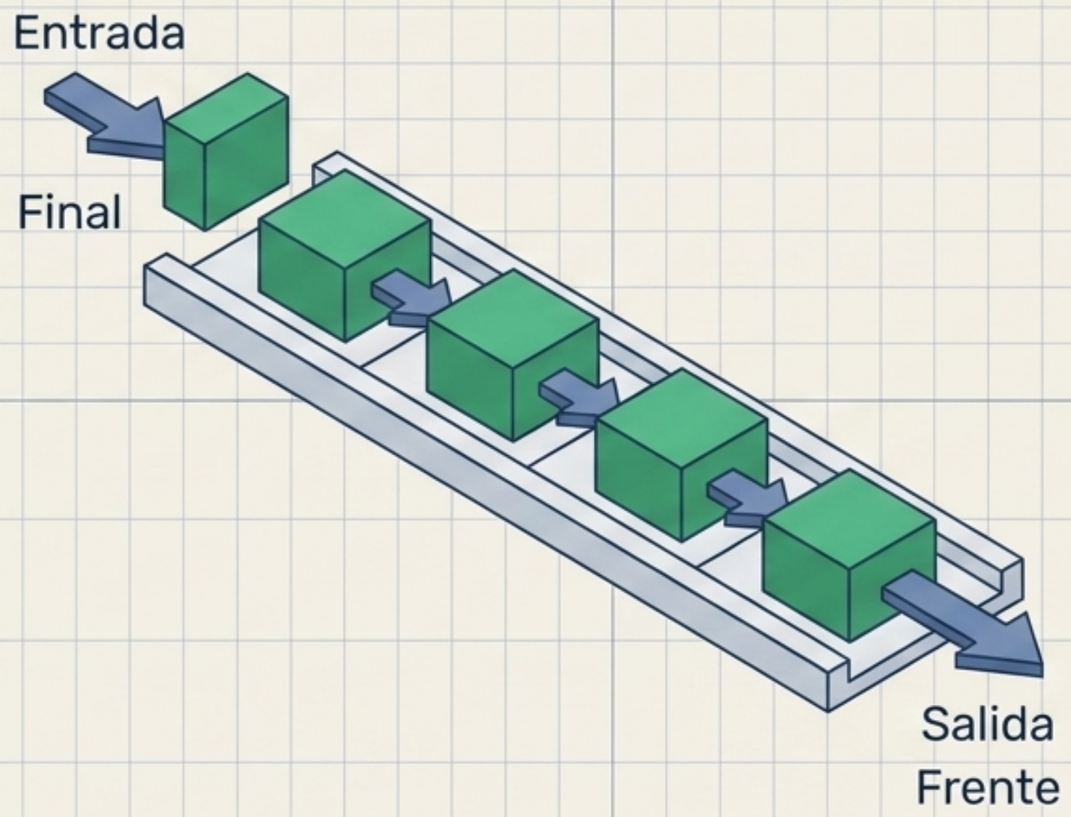
NotebookLM

Colas (Queues): El Principio FIFO

Orden estricto: Primero en entrar, primero en salir (First In, First Out). La inserción y la extracción ocurren en extremos opuestos de la estructura. La lectura también es destructiva.

CASOS DE USO

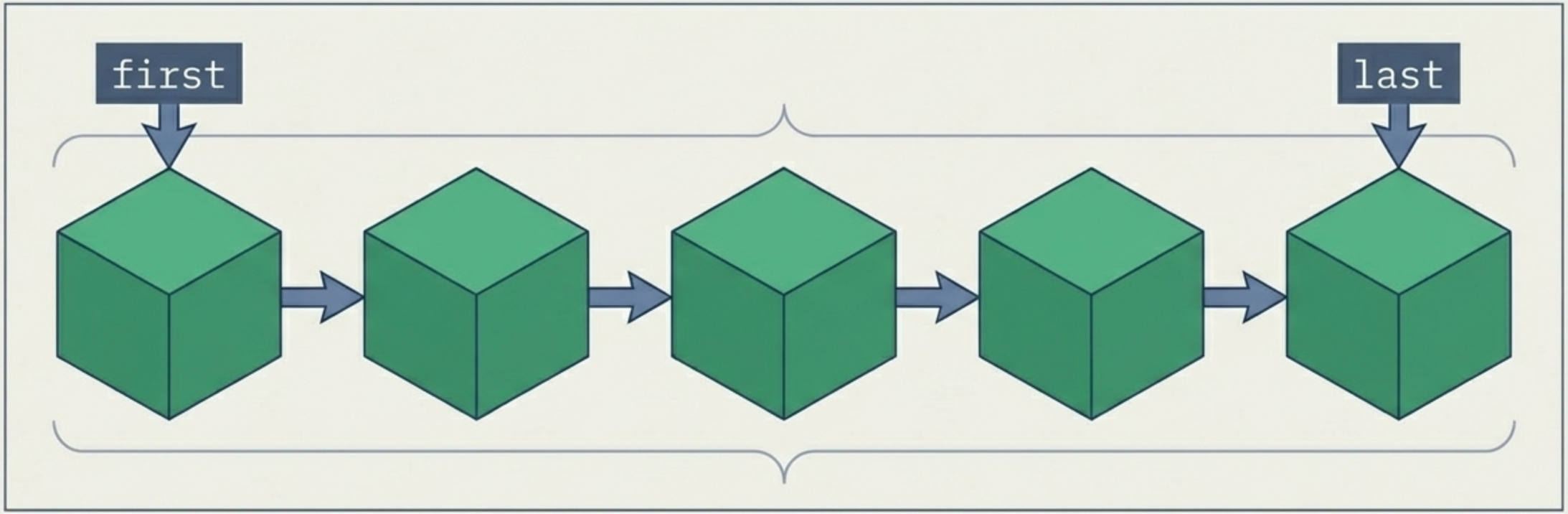
- Fila de banco virtual
- Lista de trabajos en el procesador (print spooler)



NotebookLM

Anatomía de una Cola: Dos Punteros

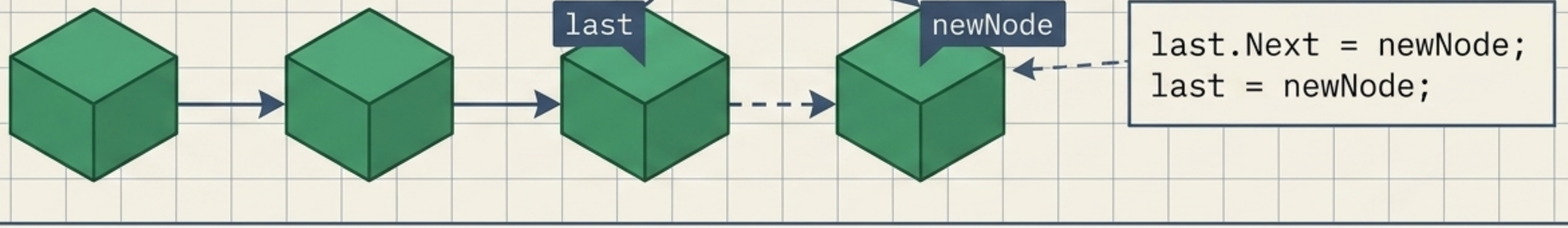
A diferencia de la pila que solo rastrea el tope, una cola requiere monitorear dos extremos simultáneamente para ser eficiente: el principio (first) para procesar extracciones, y el final (last) para enlazar nuevas inserciones.



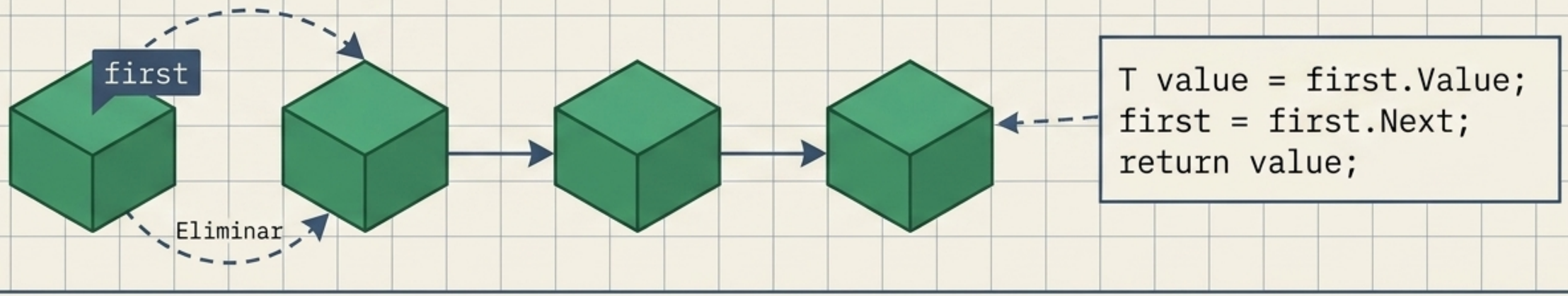
NotebookLM

Operaciones de Cola: Enqueue & Dequeue

1. ENQUEUE (Agregar)



2. DEQUEUE (Eliminar)



NotebookLM

Matriz de Síntesis: Pilas vs. Colas

Característica	Pila (Stack)	Cola (Queue)
Orden	LIFO (Último en entrar, primero en salir)	FIFO (Primero en entrar, primero en salir)
Inserción	Cabecera (Tope)	Final
Extracción	Cabecera (Tope)	Frente (Principio)
Punteros Necesarios	Uno (**head**)	Dos (**first, last`**)
Lectura	Destructiva (Pop)	Destructiva (Dequeue)

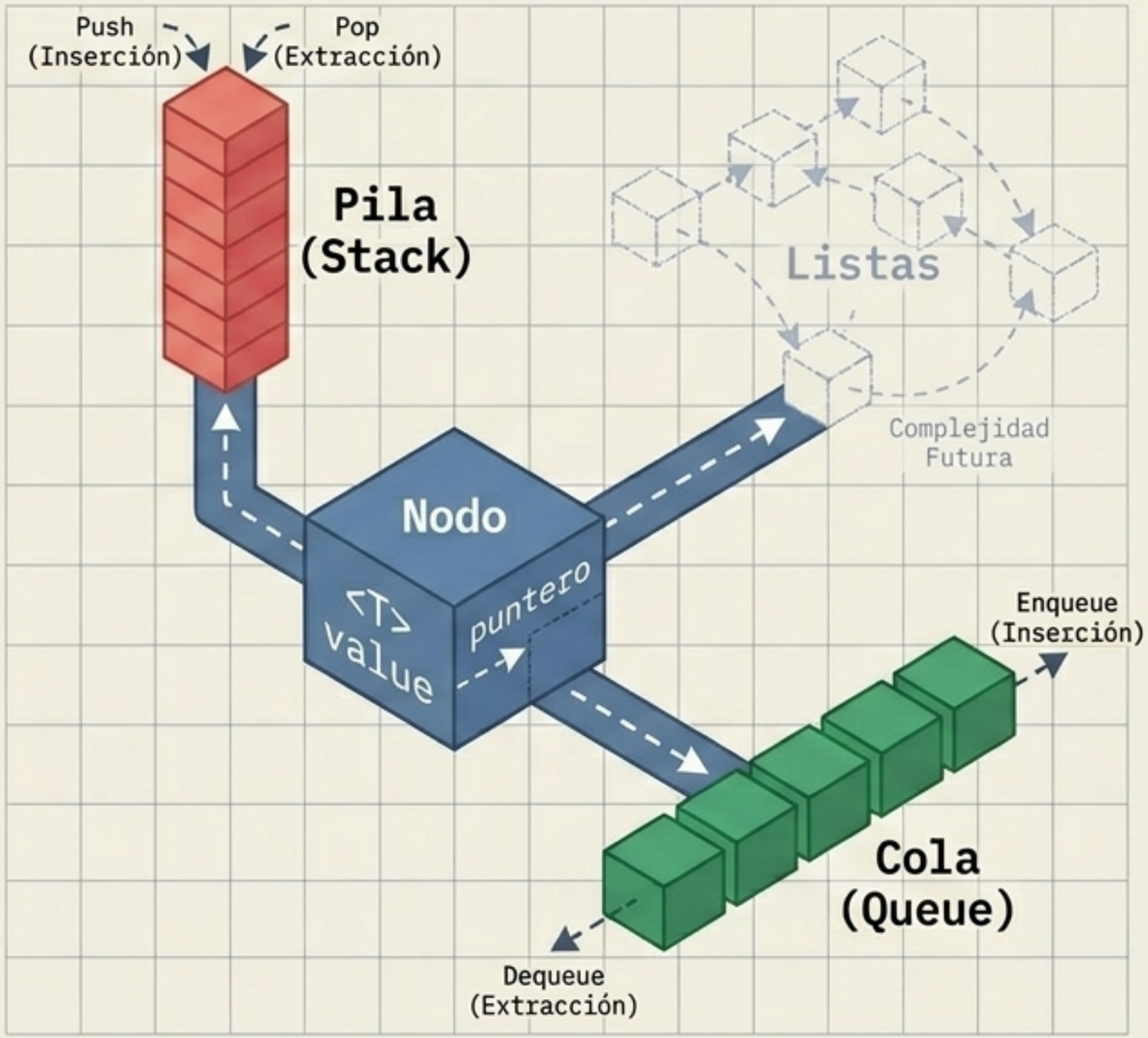
NotebookLM

Arquitectura Dinámica: Conclusiones

1 El **Heap es el lienzo**. Permite flexibilidad infinita de memoria frente a las restricciones de los vectores estáticos.

2 **Los Nodos son la base.** Combinando un valor genérico (<T>) y un puntero de referencia, se puede construir cualquier estructura.

3 **Las reglas definen la forma.** Restringir dónde ocurren las inserciones y extracciones (LIFO vs FIFO) transforma una simple cadena de nodos en herramientas lógicas específicas.



NotebookLM